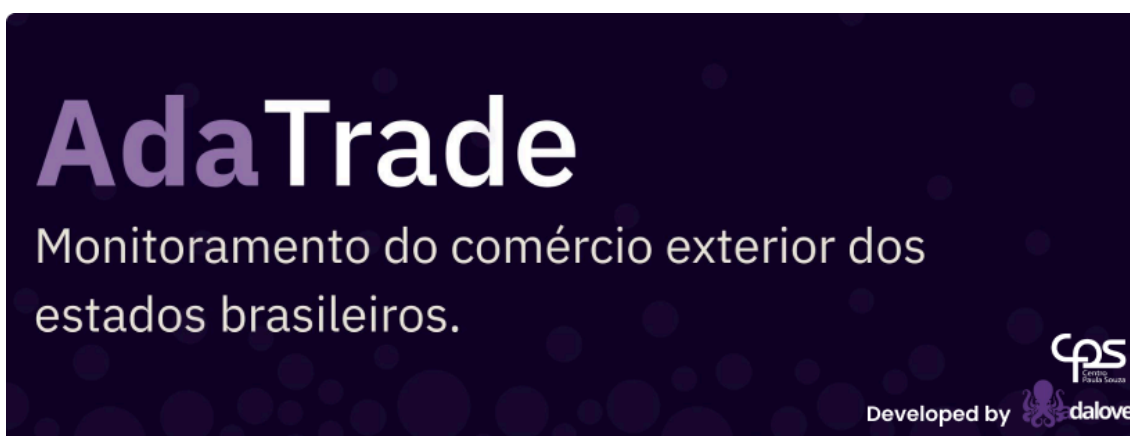


PROJETO API - SEMESTRE 3

Equipe Adalove



Descrição: Desenvolvimento de uma plataforma web que disponibilize informações sobre o desempenho dos Estados Brasileiros no comércio exterior, com base nos dados abertos do Ministério do Desenvolvimento, Indústria, Comércio e Serviços. Essa ferramenta fornecerá aos tomadores de decisão dados claros e acessíveis, permitindo a identificação de municípios que estejam em ascensão, estagnação ou declínio no mercado internacional.

Acessar projeto no Github



Equipe AdaLove

Fatec São José dos Campos – Professor Jessen Vidal

Sumário

1. Visão Geral -----	4
2. Requisitos -----	6
3. Arquitetura de Software -----	8
4. Design da Interface -----	16
5. Guia de Instalação e Configuração -----	18
6. Histórico de Versões (Changelog) -----	21
7. Limpeza dos Dados -----	23
8. Referências e Glossário -----	26



Equipe AdaLove

Fatec São José dos Campos – Professor Jessen Vidal

Sobre a Adalove

A organização foi fundada por estudantes da Faculdade de Tecnologia – Prof. Jessen Vidal, do curso Análise e Desenvolvimento de Sistemas

👤 **Missão:** Desbravar os horizontes da inovação tecnológica, oferecendo soluções eficazes que impulsionam o sucesso e a transformação digital. Com uma abordagem ágil e apaixonada, nossa equipe busca não apenas resolver problemas, mas também inspirar mudanças positivas.

👁️ **Visão:** Ser reconhecida como uma líder inovadora no campo da tecnologia. Nossa visão vai além de simplesmente fornecer soluções tecnológicas; almejamos criar um ecossistema onde a inovação seja o motor do progresso. Enxergamos um futuro onde a Adalove é sinônimo de excelência em soluções tecnológicas, inspirando confiança e impulsionando o sucesso de nossos clientes em todos os setores.

💖 **Valores:** Inovação; Colaboração; Qualidade e Paixão!

Membro	Função
Renan Tomasi	Master
Matheus Germano	Product Owner
Raphaela Barbosa	Desenvolvedor
Maria Fernanda Hansen	Desenvolvedor
Angelina Borroni	Desenvolvedor
Vitor Ribeiro	Desenvolvedor
Ramon	Desenvolvedor



Equipe AdaLove

Fatec São José dos Campos – Professor Jessen Vidal

1. Visão Geral

1.1. Aprendizagem Por Projeto Integrado (API)

Projeto pedagógico alicerçado na Metodologia API para ensino-aprendizado focado no desenvolvimento de competências e fundamentada nos pilares de aprendizado com problemas reais (RPBL), validação externa e mentalidade ágil.

Uso de estratégias para entender o problema, conceber uma solução viável ao desenvolver e implementar o MVP seguido de sua operação (CDIO).

Os resultados dos projetos devem obedecer ao Aviso Legal disponível no site da [Fatec SJC](#) com definição das datas do kickoff e das sprints.

1.2. Metodologia

O método Scrum é um método ágil adaptativo, iterativo, flexível e eficaz. Este método possui os seguintes princípios norteadores:

- Controle empírico
- Auto-organização
- Colaboração
- Priorização da criação de valor
- Time-boxing
- Desenvolvimento iterativo Entre as ferramentas utilizadas no Scrum, uma é a divisão do projeto em Sprints.

Para selecionar quais seriam as entregas das nossas Sprints, primeiro definimos nosso MVP, priorizando as tarefas que trariam maior entrega de valor para o cliente. Então, a partir das Tarefas foi construído o Backlog do Produto, o qual foi aprovado pelo cliente e dividido em 4 Backlog de Sprint.

[Clique para visualizar os Backlogs](#)

1.3. Cronograma

Sprint	Data
Kick-Off	24/02 - 28/02
1º Sprint	10/03 - 30/03
Sprint Review / Planning	31/03 - 04/04
2º Sprint	07/04 - 27/04
Sprint Review / Planning	28/04 - 02/05
3º Sprint	05/05 - 25/05
Sprint Review	26/05 - 30/05
Feira de Soluções	17/06

MVP (Minimal Valuable Product)

AdaTrade é uma plataforma que permitirá que *economistas, gestores públicos e empresas* acompanhem e analisem o desempenho dos estados brasileiros no comércio exterior de maneira eficiente e acessível.

Descrição do Projeto: O AdaTrade é uma plataforma web que visa organizar e estruturar os dados de comércio exterior brasileiro, permitindo o acompanhamento detalhado do desempenho dos estados no mercado internacional. O sistema se baseia nos dados abertos do Ministério do Desenvolvimento, Indústria, Comércio e Serviços, oferecendo uma interface interativa para análise e tomada de decisão.

2. Requisitos

2.1. Requisitos Funcionais

- **Segmentação por Estado:**

A plataforma deverá permitir a navegação por unidades federativas, exibindo informações detalhadas de cada município, como produtos exportados/importados, volume de transações e variações históricas.

- **Busca e Filtros:**

O sistema deve oferecer funcionalidades de busca por código NCM (Nomenclatura Comum do Mercosul), além de filtros personalizáveis que possibilitem análises específicas por estado, período, tipo de carga, entre outros.

- **Integração com Dados Públicos do Governo:**

A plataforma deverá integrar-se com bases públicas oficiais, especialmente os dados fornecidos pelo Ministério do Desenvolvimento, Indústria, Comércio e Serviços (MDIC), garantindo atualização e confiabilidade das informações.

- **Exportação e Manipulação de Dados:**

Os usuários devem ter a opção de exportar os dados filtrados para formatos compatíveis com ferramentas como Power BI, possibilitando análises avançadas e construção de relatórios personalizados.

- **Análises Incorporadas (Dashboards):**

O sistema deverá apresentar dashboards interativos

2.3. Requisitos Não Funcionais



Equipe AdaLove

Fatec São José dos Campos – Professor Jessen Vidal

- **Frontend:**

A interface da aplicação será desenvolvida utilizando as seguintes tecnologias:

- **React.js:** biblioteca para construção de interfaces interativas.
- **Vite:** bundler moderno e rápido para desenvolvimento web.
- **TypeScript:** linguagem que adiciona tipagem estática ao JavaScript.
- **Tailwind CSS:** framework utilitário para estilização responsiva.
- **Shcn/UI:** biblioteca de componentes acessíveis e estilizados para React.

- **Backend:**

A lógica de negócios será implementada com o framework NestJS, que oferece uma arquitetura escalável, modular e baseada em boas práticas do Node.js.

- **Banco de Dados:**

O armazenamento dos dados será feito com o PostgreSQL,

- **Pipeline de Dados:**

A limpeza, normalização e transformação dos dados será realizada por meio de notebooks no Google Colab, utilizando Python e Pandas.

3. Arquitetura do Sistema

3.1. Arquitetura do Sistema

O sistema desenvolvido é uma aplicação web modular, composta por duas grandes camadas: **Frontend** e **Backend**, comunicando-se por meio de **requisições HTTP REST**. A aplicação tem como objetivo fornecer uma plataforma de visualização e análise de dados de comércio exterior brasileiro (exportações e importações).

3.1.1. Arquitetura Geral: Camadas

Camada de Apresentação (Front-end);

Camada de Aplicação (Back-end);

Camada de Persistência de Dados;

3.1.1.1. Camada de Apresentação (Front-end)

O frontend é uma **Single Page Application (SPA)** construída com **ReactJS**. Ele é responsável pela interface com o usuário, realizando:

- Consumo da API via **axios**.
- Renderização de gráficos e tabelas para exibição dos dados.
- Navegação dinâmica entre páginas usando **React Router DOM**.
- Sistema de filtros para consultas personalizadas baseadas em Estado, Ano e NCM.

O projeto é modularizado em componentes reutilizáveis, promovendo fácil manutenção e escalabilidade.



Equipe AdaLove

Fatec São José dos Campos – Professor Jessen Vidal

3.1.1.2. Camada de Aplicação (Back-end)

O backend é uma **API RESTful** desenvolvida em **NestJS**, aplicando conceitos de modularidade e injeção de dependência. Suas responsabilidades incluem:

- Gerenciamento dos dados de Exportações, Importações, NCMs, Estados e Municípios.
- Disponibilização de endpoints para consultas dinâmicas.
- Utilização do **Prisma ORM** para abstrair as operações no banco de dados.
- Implementação de DTOs para padronizar as requisições e respostas.

A arquitetura favorece a separação clara entre camadas de Controller, Service e Repository, aumentando a organização e a testabilidade do código.



Equipe AdaLove

Fatec São José dos Campos – Professor Jessen Vidal

3.1.1.3. Camada de Persistência de Dados

A camada de persistência de dados é implementada utilizando o sistema gerenciador de banco de dados relacional PostgreSQL, com gerenciamento realizado via Docker, proporcionando facilidade na configuração, replicação e escalabilidade do ambiente.

Sua estrutura é composta por duas tabelas principais: Exportação e Importação, que armazenam respectivamente os dados detalhados sobre as exportações e importações brasileiras. Essas tabelas centralizam informações como Estado, Ano, Código NCM, Valor FOB e País de destino ou origem.

Além disso, há tabelas auxiliares que permitem normalizar os dados e garantir integridade referencial. Estas tabelas são utilizadas para otimizar buscas, aplicar filtros personalizados e evitar redundância de informações.

O acesso e manipulação dos dados são realizados através do Prisma ORM, que gera consultas SQL de forma segura e eficiente, integrando os modelos definidos no arquivo schema.prisma com a lógica da aplicação. Essa camada é essencial para garantir persistência, integridade e performance na manipulação dos grandes volumes de dados tratados pela API.



Equipe AdaLove

Fatec São José dos Campos – Professor Jessen Vidal

3.2. Fluxo do Sistema

O fluxo de funcionamento do sistema inicia-se no frontend, onde o usuário interage com a aplicação e define filtros ou parâmetros de busca, como ano, estado ou código NCM. A partir dessa interação, o frontend realiza chamadas HTTP, especificamente do tipo **GET**, para a API disponibilizada pelo backend. Essas requisições transportam os parâmetros definidos pelo usuário, solicitando os dados correspondentes.

Ao receber a requisição, o backend — desenvolvido em NestJS — interpreta os parâmetros e aciona os serviços internos responsáveis por gerenciar a lógica de acesso aos dados. Utilizando o **Prisma ORM**, o backend consulta o banco de dados relacional, aplicando os filtros recebidos para buscar exatamente as informações requeridas. Após a recuperação dos dados, o backend realiza um tratamento para formatá-los de maneira padronizada e os envia de volta ao front-end como resposta.

Com os dados recebidos, o frontend processa a resposta e organiza a apresentação das informações de forma visual e intuitiva para o usuário. Esses dados podem ser exibidos por meio de gráficos, tabelas ou outros componentes visuais, proporcionando uma análise clara e interativa dos dados de exportações e importações brasileiras.

3.3. Trade-Offs

Os trade-offs do projeto envolvem diversas decisões que afetam a qualidade, desempenho, custo e usabilidade do produto.

Separação Frontend e Backend

Decisão: Separar a aplicação em dois repositórios distintos, um para o frontend e outro para o backend, comunicando-se via API RESTful.

Trade-off:

- **Vantagem:** Facilita o desenvolvimento paralelo das equipes, melhora a organização do projeto e torna a manutenção futura mais simples, permitindo mudanças independentes em cada camada.
 - **Desvantagem:** Requer configuração adicional para integração e implantação em ambiente de produção, além de aumentar a complexidade do ambiente de desenvolvimento.
-

Simplicidade vs. Robustez na Comunicação

Decisão: Utilizar apenas requisições HTTP do tipo GET para buscar dados da API.

Trade-off:

- **Vantagem:** Simplifica o frontend e o backend, acelerando o desenvolvimento e reduzindo a curva de aprendizado dos envolvidos no projeto.
 - **Desvantagem:** Limita a expansão futura do sistema, já que a ausência de métodos como POST, PUT e DELETE impede operações de escrita e atualização, exigindo refatoração caso novas funcionalidades sejam adicionadas.
-

Segurança vs. Rapidez de Implementação

Decisão: Não implementar autenticação ou autorização nas requisições da API.

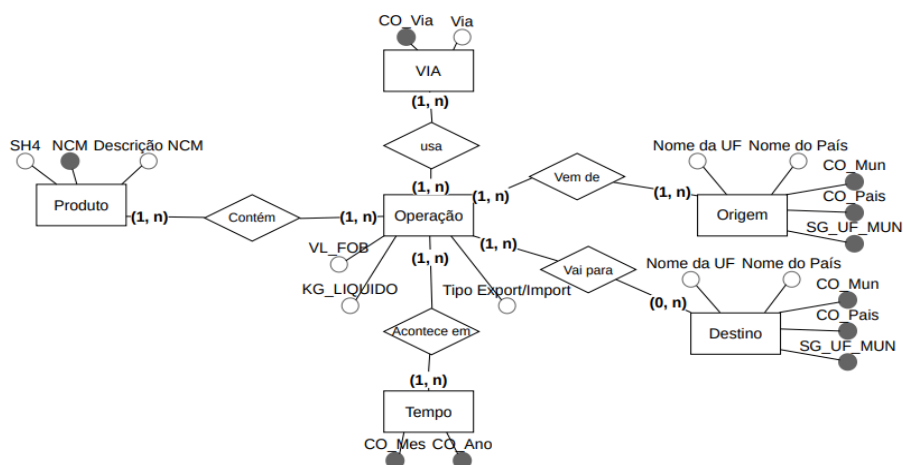
Trade-off:

Vantagem: Agiliza o desenvolvimento inicial e facilita o acesso livre aos dados para qualquer usuário.

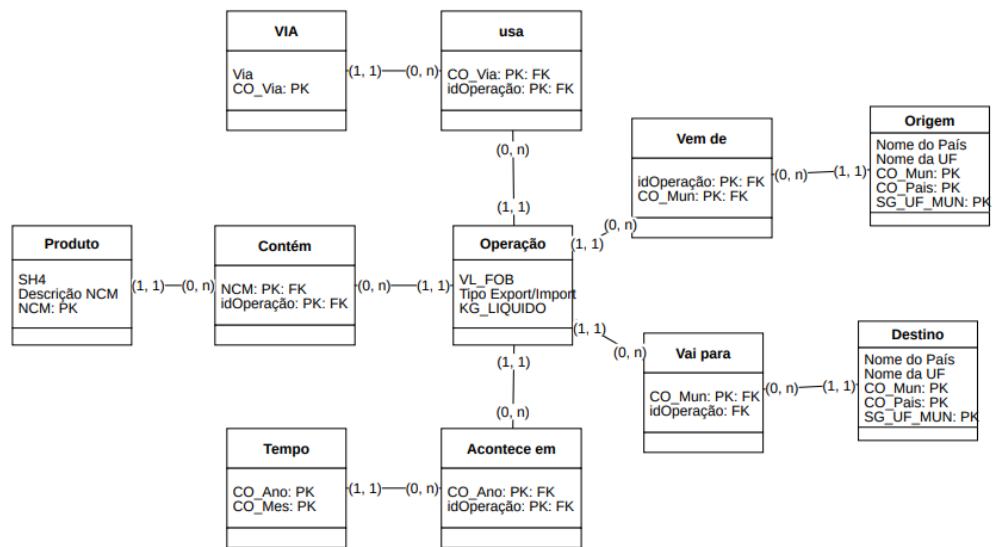
Desvantagem: Compromete a segurança da aplicação, expondo o backend a acessos indevidos e dificultando a proteção de dados em ambientes mais sensíveis ou críticos.

3.4. Diagramas

3.4.1. Modelo Conceitual



3.4.2. Modelo Conceitual



4. Design da interface

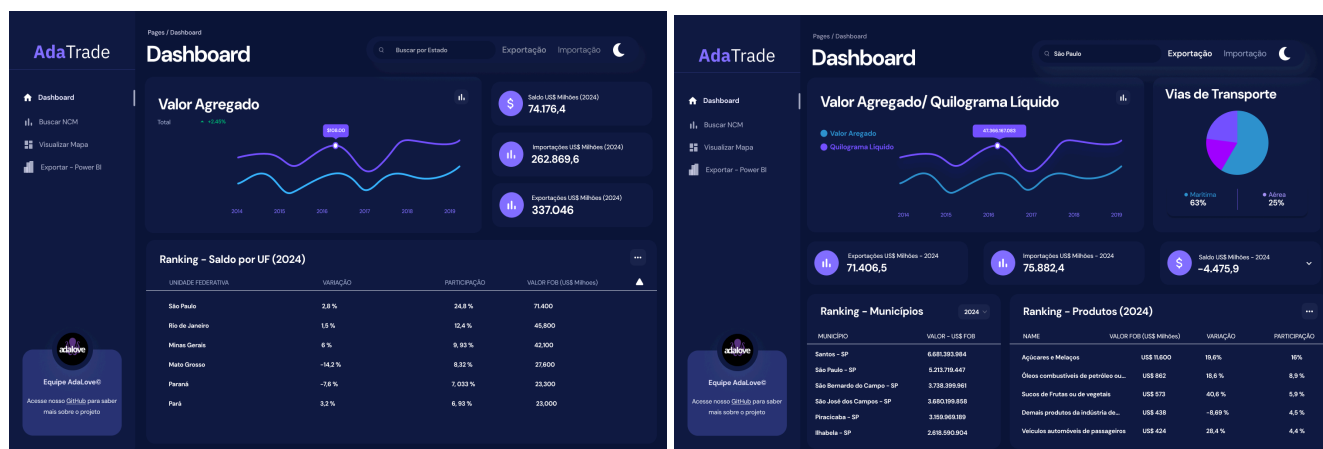
4.1. Design da Aplicação Web

A aplicação contém uma interface minimalista e intuitiva, e como principal objetivo, melhorar a experiência do usuário final.

🎨 O time de design da equipe optou por uma paleta de cores em tons de roxo, visando o minimalismo e uma aplicação limpa com interface amigável ao usuário.

4.2. Protótipo

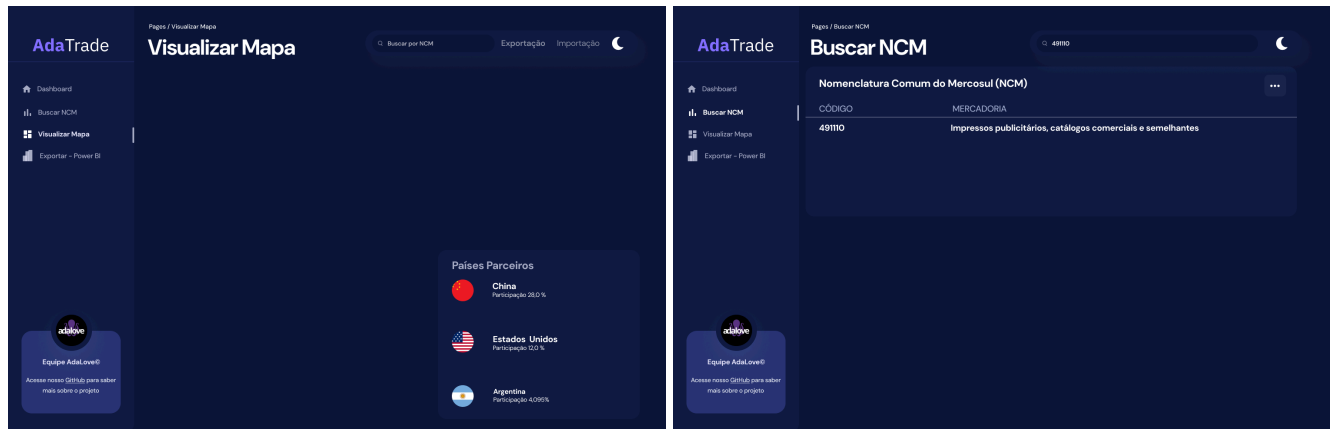
4.2.1 Modo Escuro



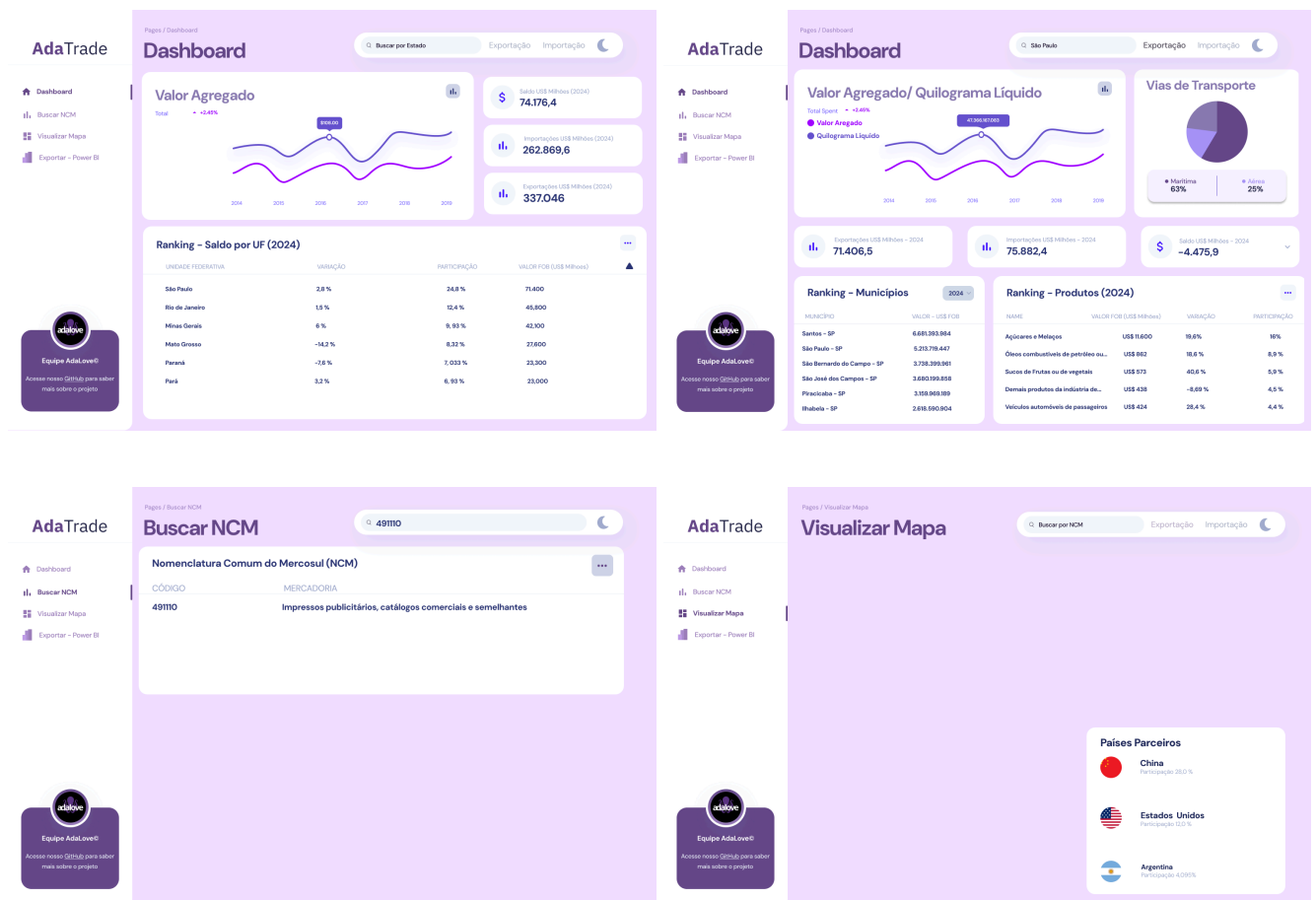


Equipe AdaLove

Fatec São José dos Campos – Professor Jessen Vidal



4.2.2 Modo Claro



5. Guia de Instalação e Configuração

5.1. Requisitos

Antes de iniciar, certifique-se de ter instalado em sua máquina:

- Node.js (versão 16 ou superior)
- Gerenciador de pacotes: npm ou yarn
- Banco de dados: PostgreSQL
- Docker Desktop
- Git

5.2. Back-end

O back-end foi desenvolvido com o framework **NestJS**. O código está disponível no repositório:

<https://github.com/equipeAdalove/Back-API-SEMESTRE3>

5.2.1. Iniciar o banco de dados PostgreSQL com Docker

```
> docker pull msgermano/api3ads-postgres:1.0
```

```
> docker run -d --name api3ads -e POSTGRES_PASSWORD=admin -p 5433:5432  
msgermano/api3ads-postgres:1.0
```



Equipe AdaLove

Fatec São José dos Campos – Professor Jessen Vidal

5.2.2. Clone e acesse o diretório do projeto

- > `git clone https://github.com/equipeAdalove/Back-API-SEMESTRE3.git`
- > `cd Back-API-SEMESTRE3`

5.2.3. Renomeando .env-example

Mude o nome do arquivo `.env-example` para `.env` e edite as variáveis de ambiente conforme necessário. Aqui será acessado o banco de dados configurado no *Docker*

Exemplo:

```
DATABASE_URL="postgresql://[DATABASE_USER]:[DATABASE_PASSWORD]@localhost:5433/[DATABASE_NAME]?schema=public"
```

```
PORT=3000
```

5.2.4. Instalar as dependências e Iniciar o servidor

- > `npm install`
- > `npx prisma generate`
- > `npm run start`



Equipe AdaLove

Fatec São José dos Campos – Professor Jessen Vidal

5.3. Front-end

5.3.1. Clone e acesse o diretório do projeto

- > `git clone https://github.com/equipeAdalove/Front-API-SEMESTRE3.git`
- > `cd Front-API-SEMESTRE3`

5.3.2. Instalar as dependências e Iniciar o servidor

- > `npm install`
- > `npm run dev`

6. Changelog

v0.1.0-sprint1-30/03/2025

Adicionado:

- Dados csv limpos e normalizados
- Estruturação do banco de dados
- Busca por NCM

v0.2.0-sprint2-27/04/2025

Adicionado:

- Dashboard principal
- Rotas de consulta
- Análise de projeções futuras
- Documentação do projeto

Corrigido:

- Estrutura do banco de dados

v0.30-release-sprint3-25/05/2025

Adicionado:

- Página de comparações
- Disponibilização para PowerBI
- Conexão front-end e back-end
- Manual do usuário

Corrigido:

- Visualização dos mapas
- Detalhes no Front-end

Removido:

- Hospedagem na nuvem

7. Limpeza dos Dados

7.1. Objetivo

O processo de limpeza de dados tem como propósito preparar os arquivos de exportação e importação do comércio exterior brasileiro para análise no projeto AdaTrade. A limpeza garante que os dados estejam padronizados, consistentes e livres de ruídos que possam comprometer a qualidade das análises.

7.2. Etapas da Limpeza

7.2.1. Acesso aos Dados

A primeira etapa consiste em conectar o ambiente de desenvolvimento (Google Colab) ao Google Drive. Dessa forma, é possível acessar os arquivos brutos em formato .csv e também salvar os arquivos limpos em uma pasta de destino específica.

7.2.2. Seleção de Informações Relevantes

Para assegurar que apenas dados úteis sejam considerados, foram definidas colunas obrigatórias com informações essenciais como ano, mês, código do produto (NCM), país de destino ou origem, via de transporte, unidade federativa, quantidade, peso líquido e valor FOB. Arquivos que não contivessem todas essas colunas foram automaticamente descartados.

7.2.3. Exclusão de Registros Irrelevantes

Foram removidos registros com vias de transporte consideradas não representativas para o comércio exterior em larga escala, como transporte postal, courier, em mãos, meios próprios, ou ainda aqueles com vias não declaradas ou desconhecidas. Esses dados são frequentemente genéricos, inconsistentes ou estatisticamente irrelevantes.

7.2.4. Eliminação de Dados Ausentes

Registros com valores ausentes em qualquer uma das colunas obrigatórias foram descartados. Também foram eliminadas linhas completamente vazias, garantindo que apenas dados válidos fossem processados.

7.2.5. Tratamento de Outliers

Foi aplicada uma técnica estatística chamada IQR (Intervalo Interquartil) para identificar e remover valores fora da faixa esperada em colunas numéricas. No caso dos dados de exportação, esse tratamento foi feito na coluna de peso líquido. Já nos dados de importação, a limpeza por outliers foi aplicada ao peso líquido e também aos valores de frete, seguro e FOB. Isso ajuda a evitar distorções causadas por registros anômalos ou erros de digitação.

7.2.6. Consolidação dos Arquivos

Após a limpeza, todos os arquivos válidos foram combinados em um único arquivo final para cada tipo de dado (exportação e importação). O resultado foi salvo em formato .csv e armazenado no Google Drive em uma pasta separada de dados limpos.

7.3 Resultado Final

Como resultado desse processo de limpeza, foram gerados dois arquivos prontos para análise:

- Um contendo os dados limpos de exportação.
- Outro contendo os dados limpos de importação.

Ambos os arquivos apresentam apenas registros completos, relevantes e estatisticamente confiáveis, contribuindo para uma análise mais precisa e segura no projeto AdaTrade.

8. Referências e Glossário

8.1 Referências

<https://www.gov.br/mdic/pt-br/assuntos/comercio-exterior/estatisticas/base-de-dados-bruta>

<https://comexstat.mdic.gov.br/pt/home>

8.2 Glossário

API RESTful

Interface de programação de aplicações que segue os princípios REST, permitindo a comunicação entre sistemas por meio de requisições HTTP (como GET, POST, PUT, DELETE).

NestJS

Framework de Node.js para construção de aplicações backend escaláveis e modulares, baseado em TypeScript e arquitetura orientada a módulos.

Vite

Empacotador moderno e rápido para aplicações front-end, que otimiza o processo de desenvolvimento e build de aplicações web modernas.

Tailwind CSS

Framework CSS utilitário que permite a criação rápida de interfaces responsivas, utilizando classes predefinidas diretamente no HTML.



Equipe AdaLove

Fatec São José dos Campos – Professor Jessen Vidal

Shadcn/UI

Biblioteca de componentes de interface de usuário acessíveis, personalizáveis e estilizados, geralmente utilizada em aplicações React.

Prisma ORM

ORM (Object-Relational Mapping) para Node.js e TypeScript que facilita o acesso a bancos de dados relacionais com segurança de tipos e consultas otimizadas.

DTO (Data Transfer Object)

Objeto que transporta dados entre camadas da aplicação, padronizando o formato de entrada e saída das informações nas APIs.

NCM (Nomenclatura Comum do Mercosul)

Sistema de codificação utilizado para classificar mercadorias nos países do Mercosul, essencial para fins de exportação, importação e estatísticas de comércio exterior.

SPA (Single Page Application)

Aplicação web que carrega uma única página HTML e atualiza dinamicamente o conteúdo exibido sem recarregar a página inteira, proporcionando maior fluidez na navegação.

Google Colab

Ambiente online do Google que permite executar códigos em Python por meio de notebooks interativos hospedados na nuvem, amplamente utilizado para análise e manipulação de dados.